Recommender System

Lecture 15

肖升生 博士
Dr. Shengsheng Xiao
Email: Xiao.shengsheng@shufe.edu.cn

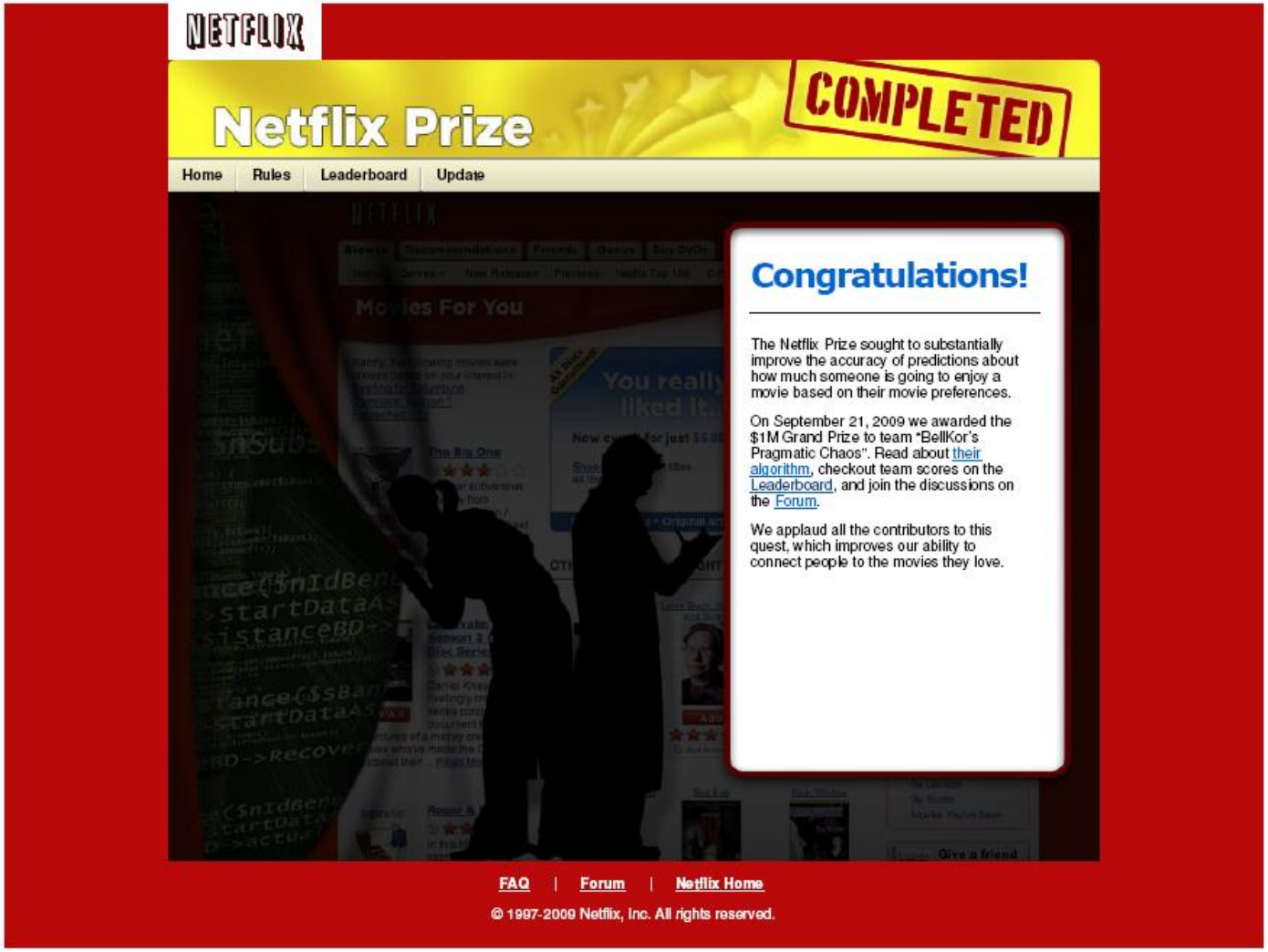
Outline

- What is recommender system
- Content-based method
- Collaborative filtering method
  - User based nearest neighbor
  - Item based nearest neighbor
  - Latent factor model
- Case study

Outline

- What is recommender system
- Content-based method
- Collaborative filtering method
  - User based nearest neighbor
  - Item based nearest neighbor
  - Latent factor model
- Case study

Netflix challenge



The Netflix Prize Contest

- **GOAL:** use *training data* to build a recommender system, which, when applied to *qualifying data*, improves error rate by 10% relative to Netflix's existing system
- **PRIZE:** first team to 10% wins $1,000,000
- **SCHEDULE:**
  - Started Oct. 2, 2006
  - To end after 5 years

The Netflix Prize Contest

- **PARTICIPATION:**
  - 51,051 contestants on 41,305 teams from 186 different countries
  - 44,014 valid submissions from 5,169 different teams

Rank	Team Name	Best Test Score	% Improvement	Best Submit Time
Grand Prize - RMSE = 0.8567 - Winning Team: BellKor's Pragmatic Chaos				
1	BellKor's Pragmatic Chaos	0.8567	10.06	2009-07-26 18:18:28
2	The Ensemble	0.8567	10.06	2009-07-26 18:38:22
3	Grand Prize Team	0.8582	9.90	2009-07-10 21:24:40
4	Opera Solutions and Vandelay United	0.8588	9.84	2009-07-10 01:12:31
5	Vandelay Industries !	0.8591	9.81	2009-07-10 00:32:20
6	PragmaticTheory	0.8594	9.77	2009-06-24 12:06:56
7	BellKor in BigChaos	0.8601	9.70	2009-05-13 08:14:09
8	Dace	0.8612	9.59	2009-07-24 17:18:43
9	Feeds2	0.8622	9.48	2009-07-12 13:11:51
10	BigChaos	0.8623	9.47	2009-04-07 12:33:59
11	Opera Solutions	0.8623	9.47	2009-07-24 00:34:07
12	BellKor	0.8624	9.46	2009-07-26 17:19:11
Progress Prize 2008 - RMSE = 0.8627 - Winning Team: BellKor in BigChaos				
13	xiangliang	0.8642	9.27	2009-07-15 14:53:22
14	Gravity	0.8643	9.26	2009-04-22 18:31:32
15	Ces	0.8651	9.18	2009-06-21 19:24:53
16	Invisible Ideas	0.8653	9.15	2009-07-15 15:53:04
17	Just a guy in a garage	0.8662	9.06	2009-05-24 10:02:54
18	J Dennis Su	0.8666	9.02	2009-03-07 17:16:17

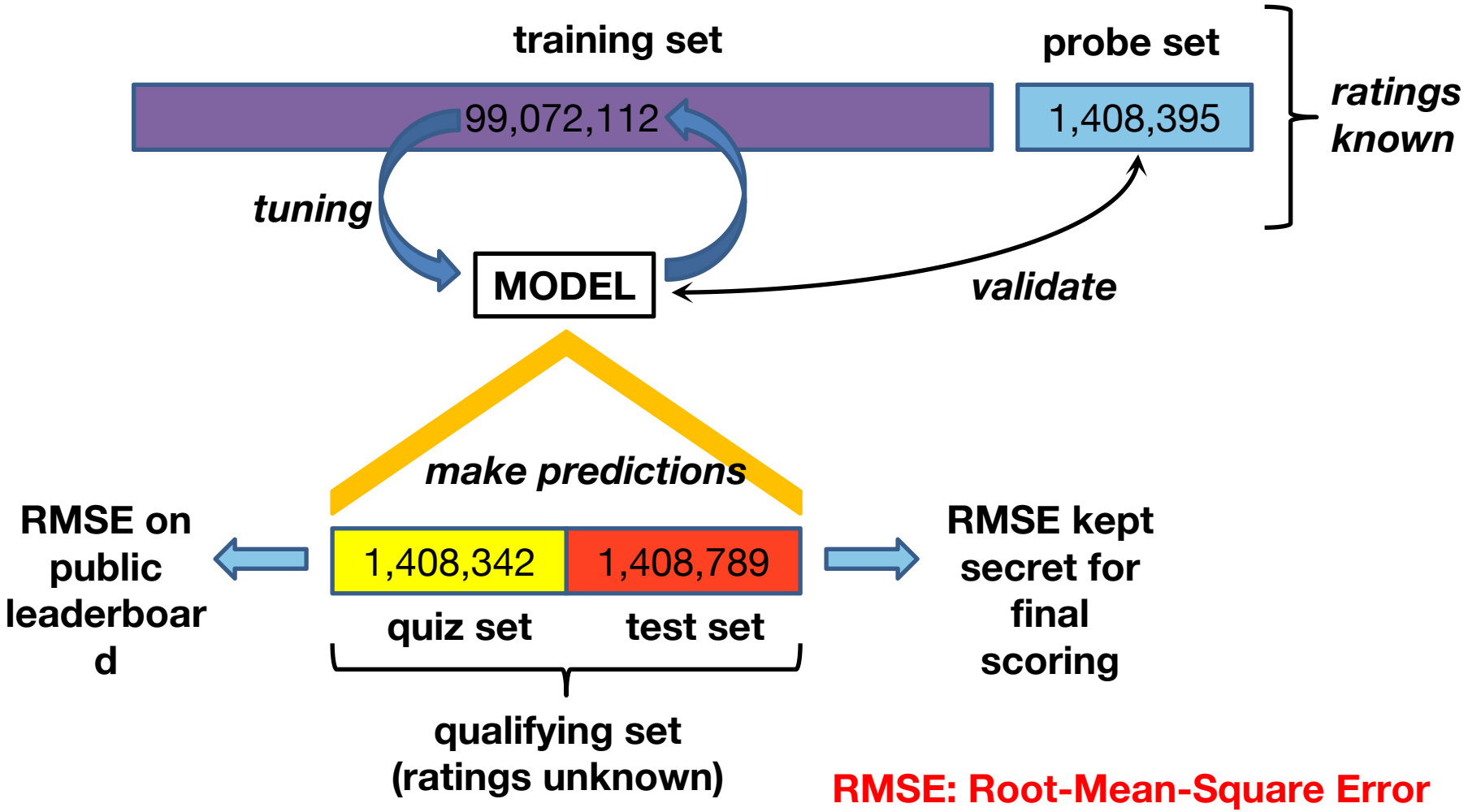
https://www.netflixprize.com/leaderboard.html

The Netflix Prize Data

- Netflix released three datasets
  - 480,189 *users* (anonymous)
  - 17,770 *movies*
  - *ratings* on integer scale 1 to 5

- *Training set*: 99,072,112 < *user, movie* > pairs with *ratings*
- *Probe set*: 1,408,395 < *user, movie* > pairs with *ratings*
- *Qualifying set* of 2,817,131 < *user*, *movie* > pairs with *no ratings*

Model and Submission Process



RMSE: Root-Mean-Square Error

Netflix Challenge

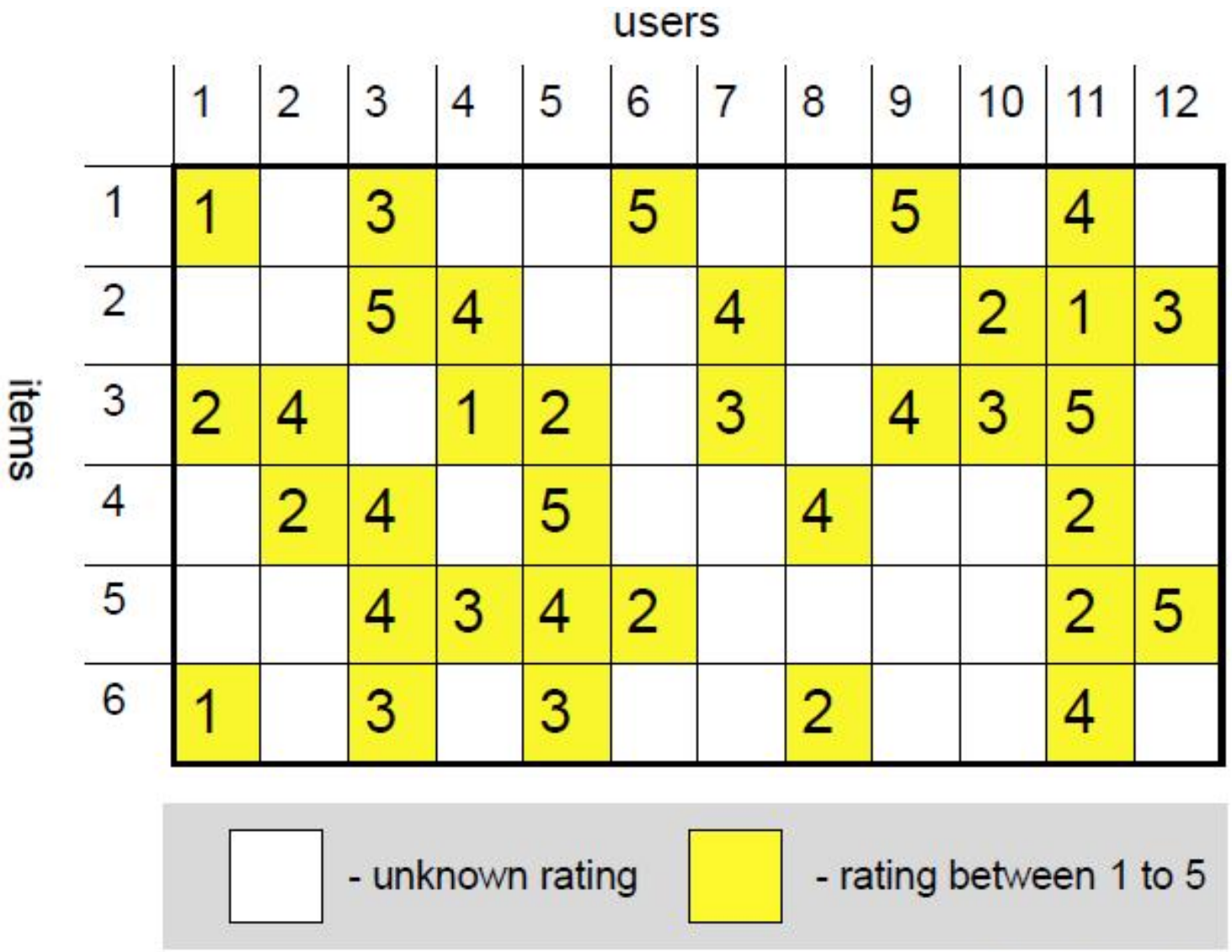
Training data

user	movie	date	score
1	21	5/7/02	1
1	213	8/2/04	5
2	345	3/6/01	4
2	123	5/1/05	4
2	768	7/15/02	3
3	76	1/22/01	5
4	45	8/3/00	4
5	568	9/10/05	1
5	342	3/5/03	2
5	234	12/28/00	2
6	76	8/11/02	5
6	56	6/15/03	4

Test data

user	movie	date	score
1	62	1/6/05	?
1	96	9/13/04	?
2	7	8/18/05	?
2	3	11/22/05	?
3	47	6/13/02	?
3	15	8/12/01	?
4	41	9/1/00	?
4	28	8/27/05	?
5	93	4/4/05	?
5	74	7/16/03	?
6	69	2/14/04	?
6	83	10/3/03	?

Rating matrix



Why the Netflix Prize Was Hard

- Massive dataset
- Very sparse – matrix only 1.2% occupied
- Extreme variation in number of ratings per user
- Statistical properties of qualifying and probe sets different from training set

	movie 1	movie 2	movie 3	movie 4	movie 5	movie 6	movie 7	movie 8	movie 9	movie 10	...	movie 17770
user 1			1		2							3
user 2		2		3	3			4				
user 3							5	3		4		
user 4	2				3			2				2
user 5		4				5			3			4
user 6			2									
user 7			2					4	2	3		
user 8	3	4				4						
user 9									3			
user 10			1		2							2
...												
user 480189		4			3			3				

Recommendation Systems

- Recommendation helps to match users with items
  - Ease information overload
  - Sales assistance (guidance, advisory, persuasion,…)

- Different system designs / paradigms
  - Based on availability of exploitable data
  - Implicit and explicit user feedback
  - Domain characteristics

Recommendation Systems

User Inf.
(Age, Gender, Tags)

Item Inf.
(name, description)

Interactions
(rating, purchasing)

Recommendation System

Item Set

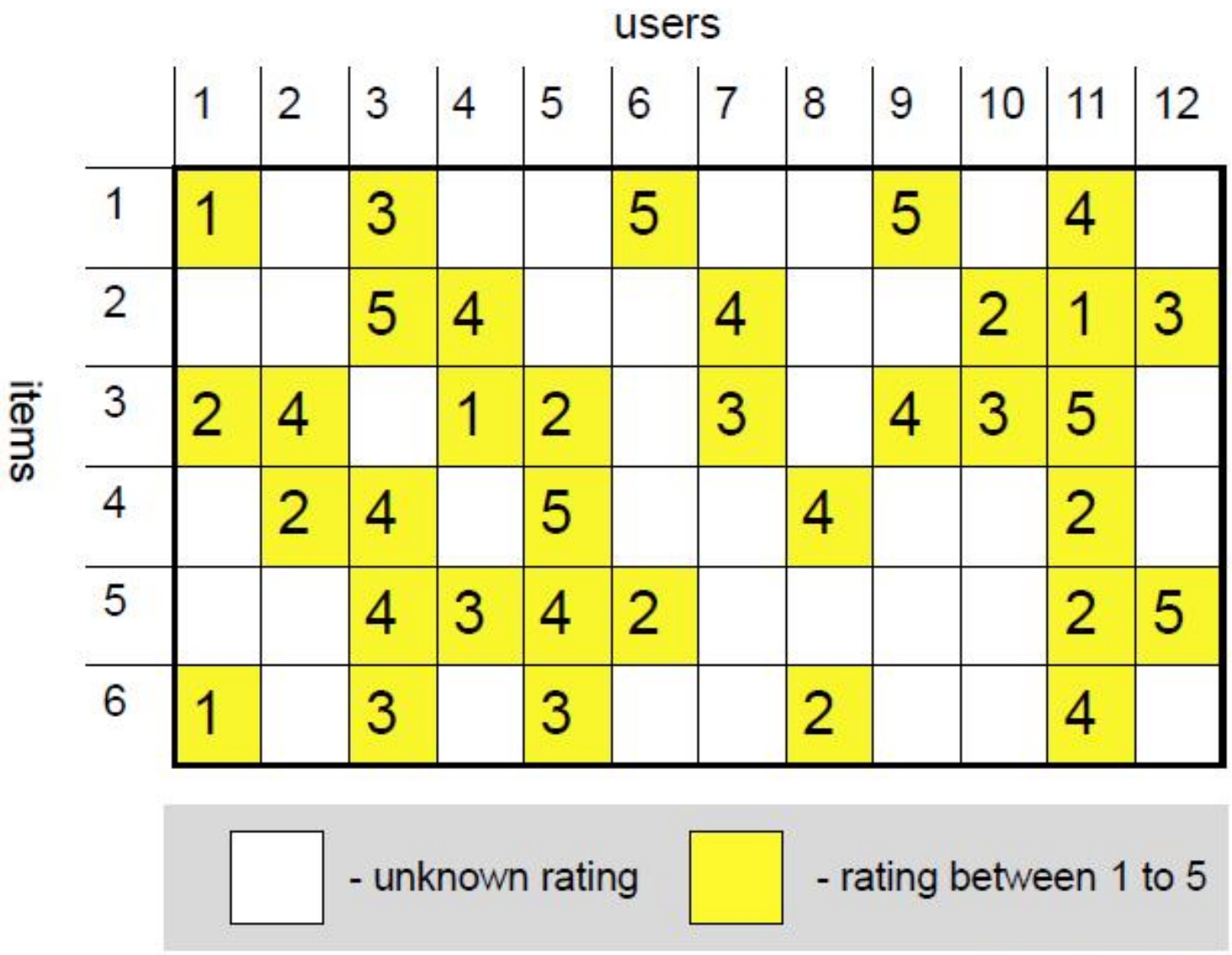
Predict users' item preference

User Base

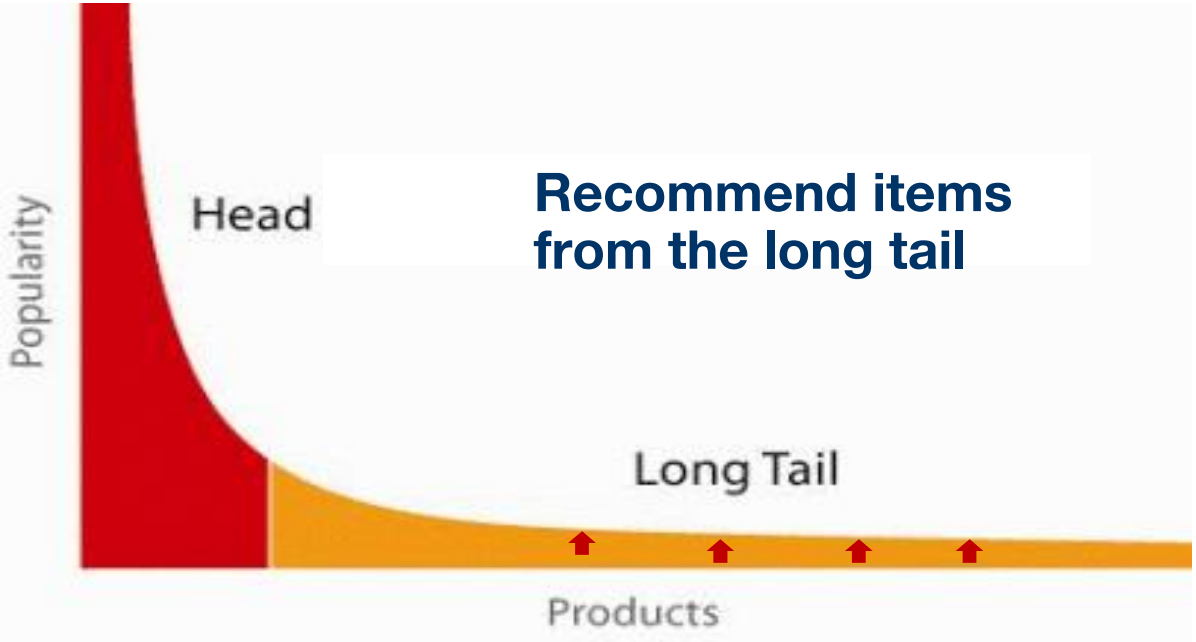
Problem Definition

- RS seen as a function
- Input:
  - User model (e.g. ratings, preferences, demographics, situational context)
  - Items (with or without description of item characteristics)
- Output:
  - Relevance score used for ranking.

Problem Definition



When does a RS do its job well?



Help users find substitute products

Levi's

Levi's Men's 501 Original Shrink To Fit Jean

★★★★☆ 1,353 customer reviews | 6 answered questions

Price: $17.33 - $59.99 & FREE Returns on some sizes and colors. Details
Sale: Lower price available on select options

Fit: As expected (71%)

Size:

Select Size Chart

Color: Rigid STF

- 100% Cotton
- Imported
- Machine Wash
- Straight-leg jean in shrink-to-fit denim that fades with repeat washings
- Five-pocket styling
- Button Fly
- 17.25-leg opening
- Actual coloration may vary from garment to garment due to specific wash processes

Customers Who Viewed This Item Also Viewed

Levi's Men's Jeans 501 Original Fit
★★★★½ 4,655
$34.99 - $64.99

Levi's Men's 505 Regular Fit Jean
★★★★☆ 5,479
#1 Best Seller in Men's Jeans
$39.49 - $68.00

Levi's Men's 514 Straight Jean
★★★★☆ 3,089
$19.15 - $58.00

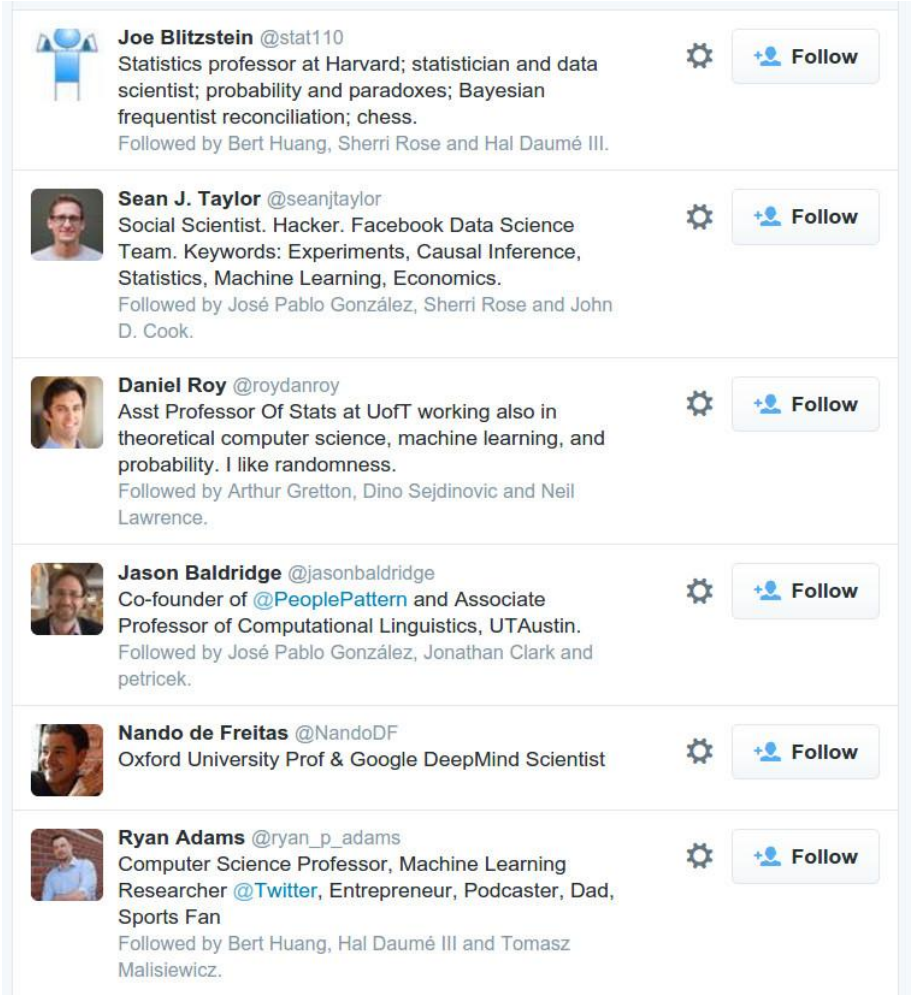
Levi's Men's Big-Tall 501 Shrink To Fit Jean
★★★★☆ 253
$39.99 - $59.99

Levi's Men's 501 Colored Rigid Shrink-to-Fit Jean (Clearance), Light Gray
★★★★☆ 63
$16.70 - $49.99

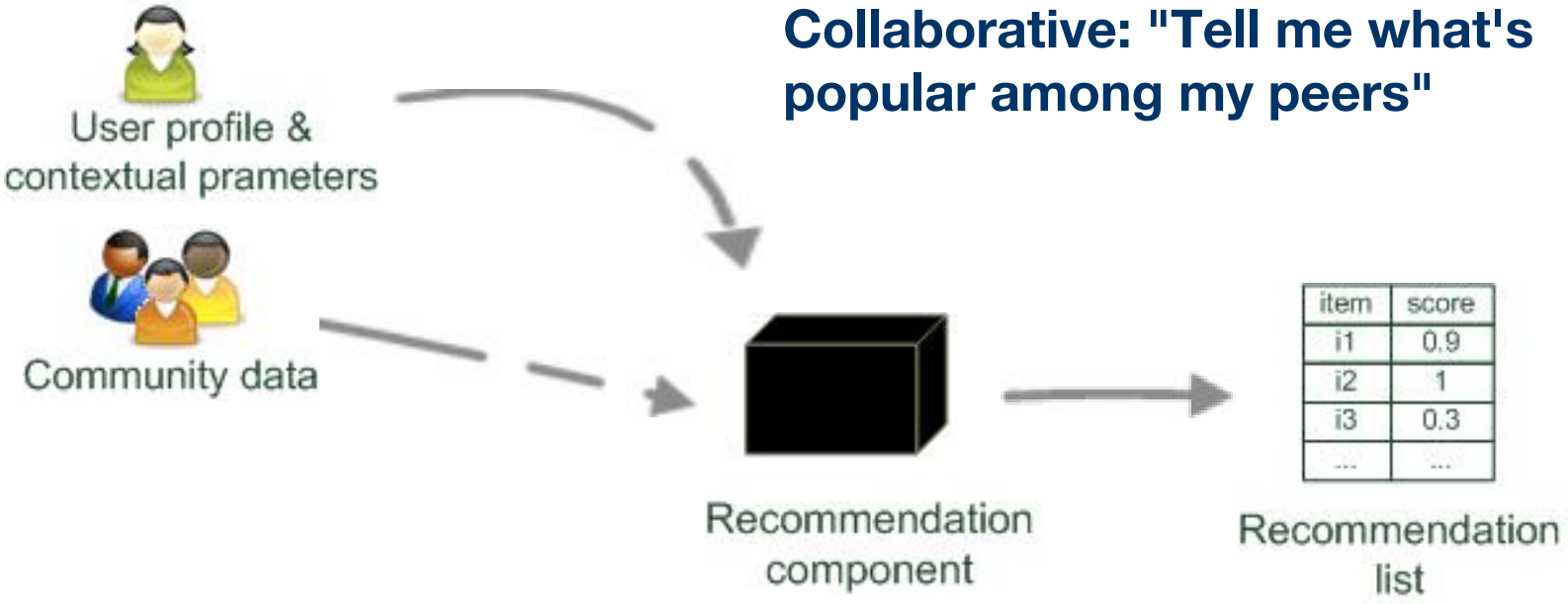
Levi's Men's 501 Colored Rigid Shrink-to-Fit Jean (Clearance), Blue Green Rigid
★★★★☆ 80
$16.83 - $49.99

Levi's Men's Big & Tall 501 Original-Fit Jean
★★★★½ 241
$46.99 - $59.99

Whom to follow

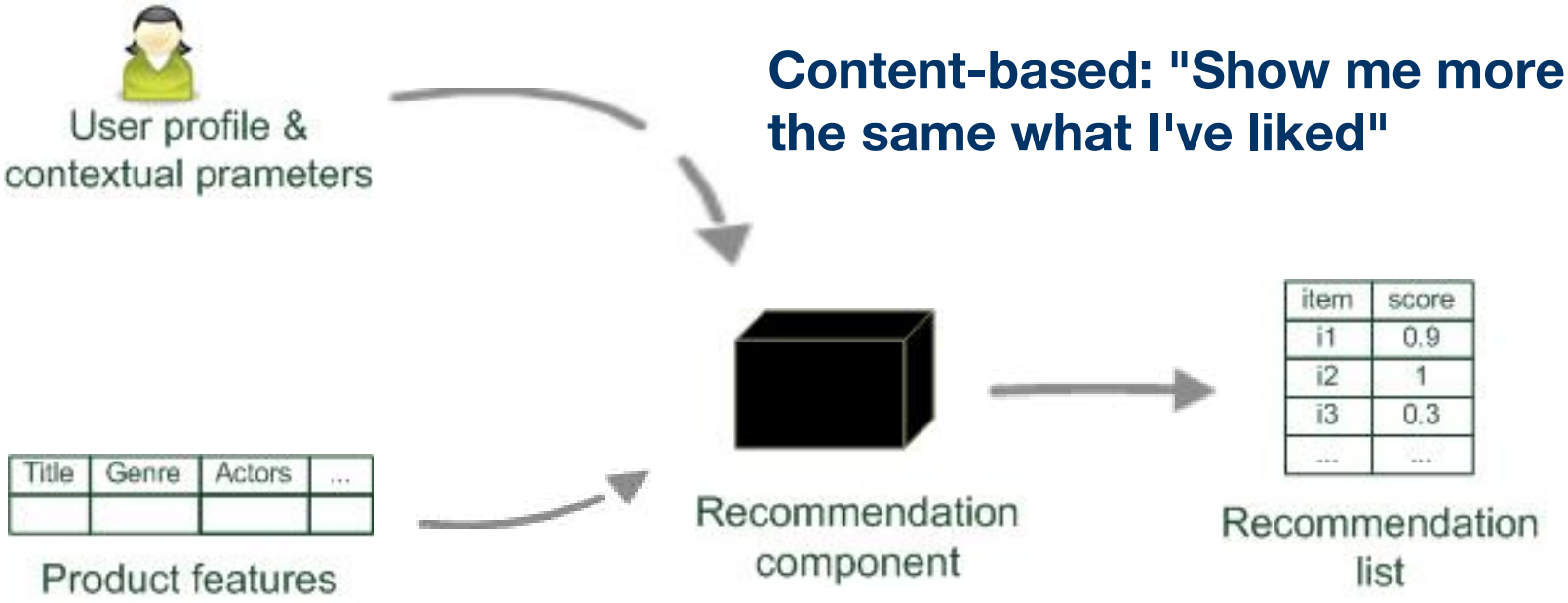


Paradigms of recommender systems

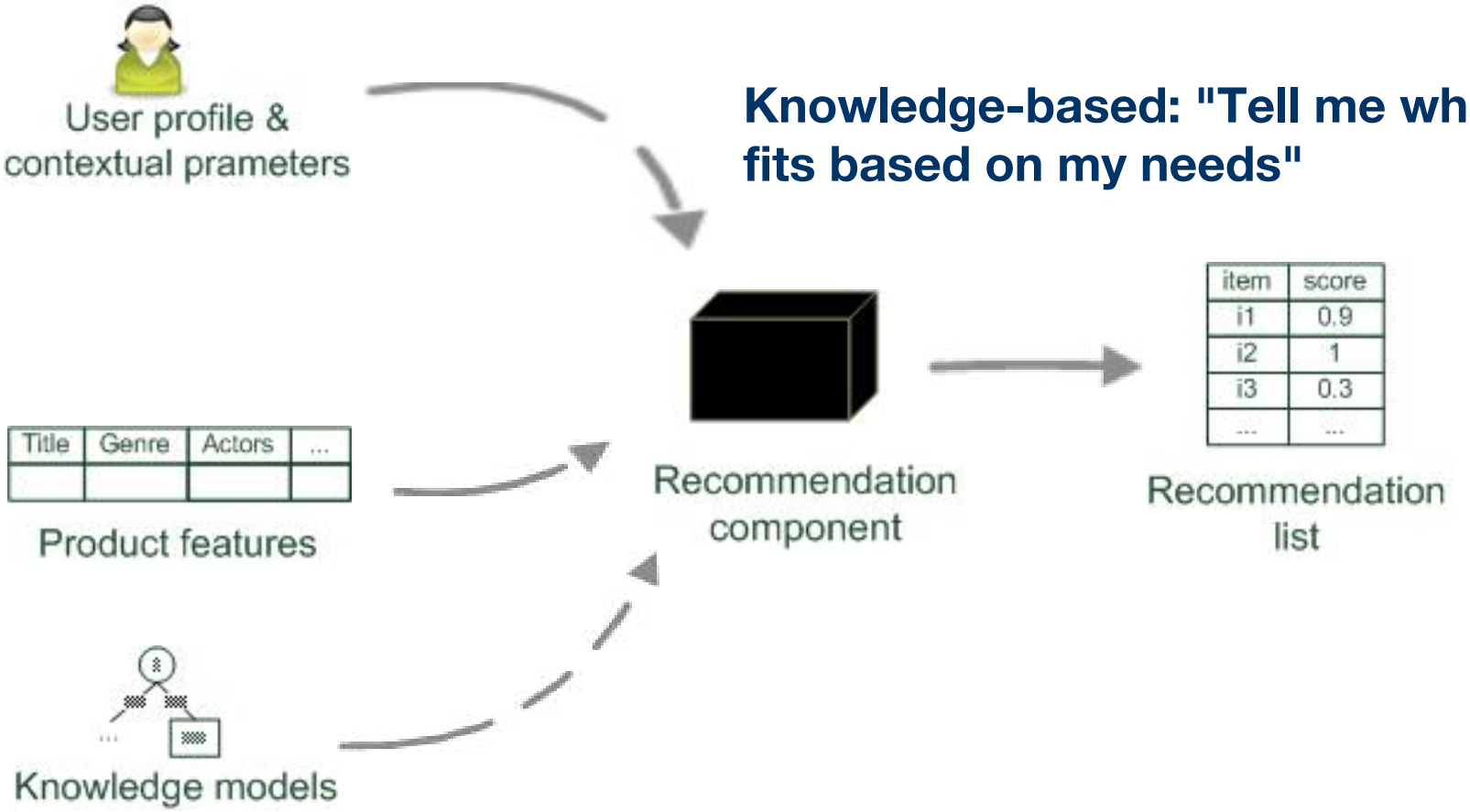


Collaborative: "Tell me what's popular among my peers"

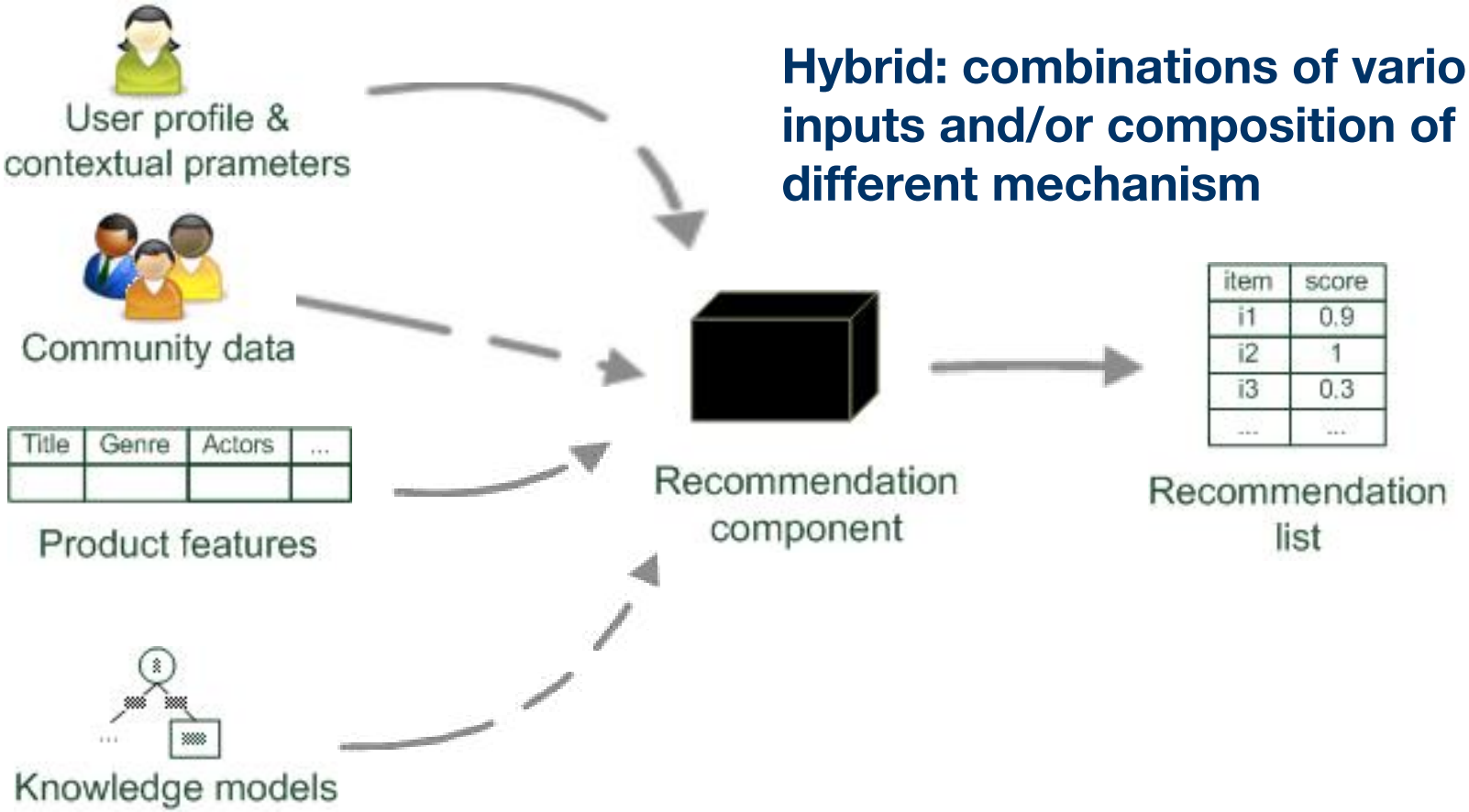
Paradigms of recommender systems



Paradigms of recommender systems

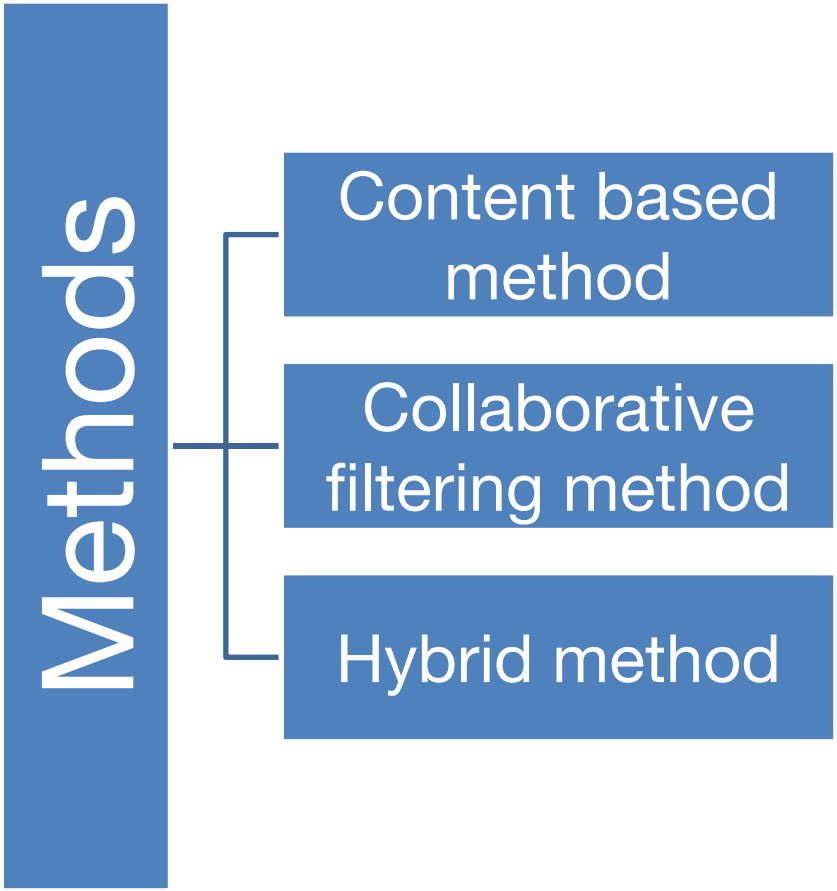


Paradigms of recommender systems



Hybrid: combinations of various inputs and/or composition of different mechanism

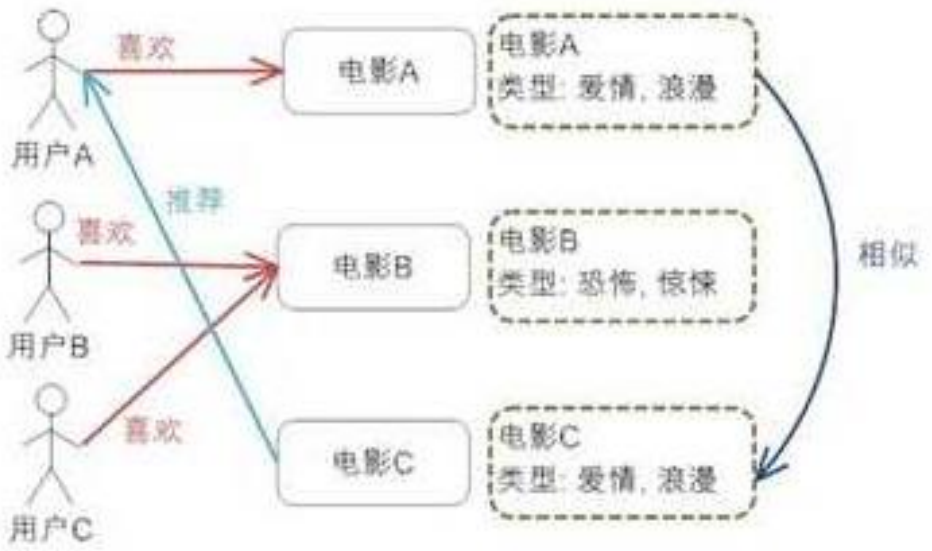
Recommendation: Methods



Outline

- What is recommender system
- Content-based method
- Collaborative filtering method
  - User based nearest neighbor
  - Item based nearest neighbor
  - Latent factor model
- Case study

Content based Recommendation



Content based Recommendation

- **Main idea:** Recommend items to customer *x* similar to previous items rated highly by *x*

Example:

- **Movie recommendations**
  - Recommend movies with same actor(s), director, genre, …
- **Websites, blogs, news**
  - Recommend other sites with "similar" content

Content-based recommendation

- **What do we need:**
  - some information about the available items such as the genre ("content")
  - some sort of *user profile* describing what the user likes (the preferences)
- **The task:**
  - learn user preferences
  - locate/recommend items that are "similar" to the user preferences
- **Solutions:**
  - Item profiles
  - User profiles/preferences
  - Recommendation list

Content based Recommendation

Item Profiles

- For each item, create an **item profile**
- **Profile is a set (vector) of features**
  - Movies: author, title, actor, director,…
  - Text: set of "important" words in document
- **How to pick important features?**
  - Usual heuristic from text mining is TF-IDF (Term frequency * Inverse Doc Frequency)
    - Term … feature
    - Document … item

Content based Recommendation

Name	Category 1	Category 2	Category 3	…
Movie 1	1	1	1	…
Movie 2	0	0	0	…
Movie 3	1	0	1	…

Similarity based recommendation

Name	Category 1	Category 2	Category 3	Like
Movie 4	1	1	1	?
Movie 5	0	0	0	?

Name	Category 1	Category 2	Category 3	Like
Movie 1	1	1	1	1
Movie 2	0	0	0	0
Movie 3	1	0	1	1

Classification-model based recommendation

Disadvantages

- –: Finding the appropriate features is hard
  - E.g., images, movies, music
- –: Overspecialization
  - Never recommends items outside user's content profile
  - People might have multiple interests
  - Unable to exploit quality judgments of other users
- –: Recommendations for new users
  - How to build a user profile?

Outline

- What is recommender system
- Content-based method
- Collaborative filtering method
  - User based nearest neighbor
  - Item based nearest neighbor
  - Latent factor model
- Case study

Collaborative Filtering

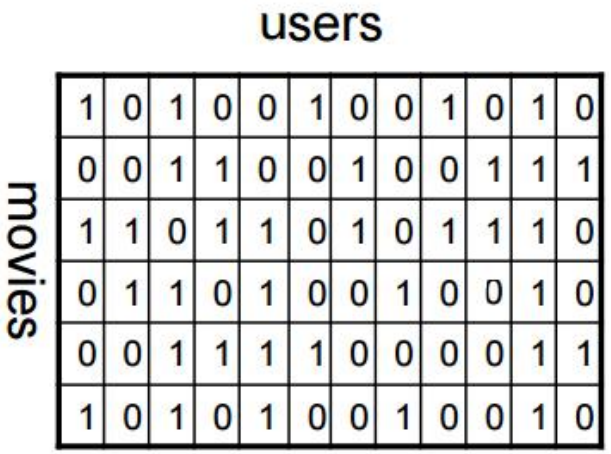
DOMAIN: some field of activity where *users* buy, view, consume, or otherwise experience *items*

PROCESS:

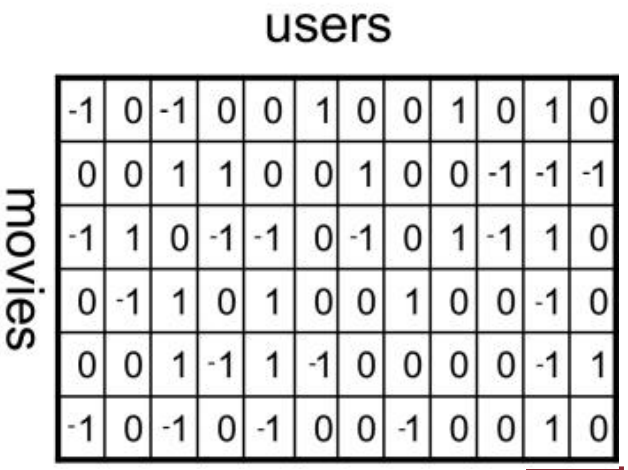
1. *users* provide *ratings* on *items* they have experienced
2. Take all < *user*, *item*, *rating* > data and build a predictive model
3. For a *user* who hasn't experienced a particular *item*, use model to *predict* how well they will like it (i.e. *predict rating*)

Other Collaborative Filtering

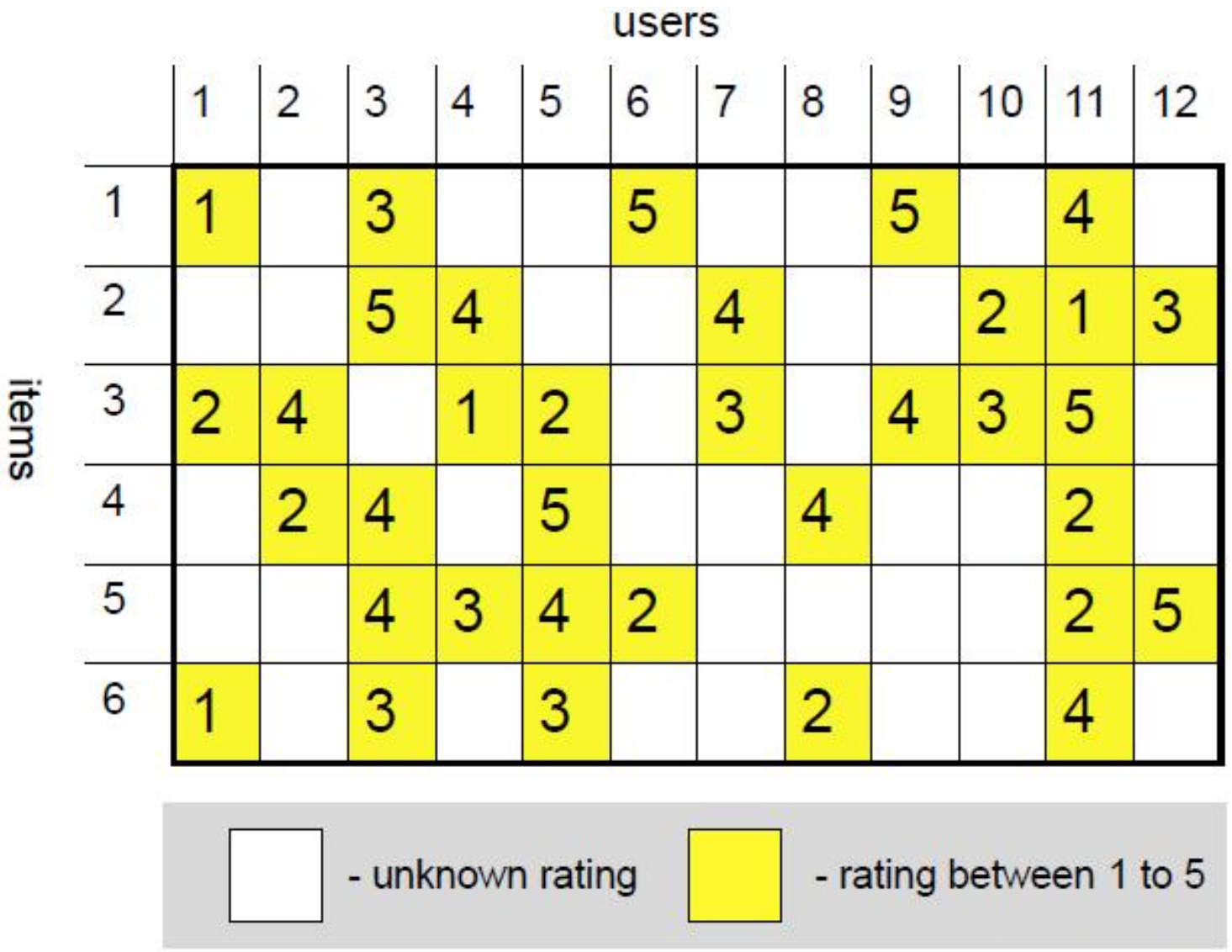
- Binary (0/1) matrix:
  - A user viewed/bought an item (1)
  - A user did not view/buy an item (0)



- 0/-1/1 matrix:
  - 1 user viewed/bought an item and liked it
  - 1 user viewed/bought an item and did not like it
  - 0 user did not view/buy an item

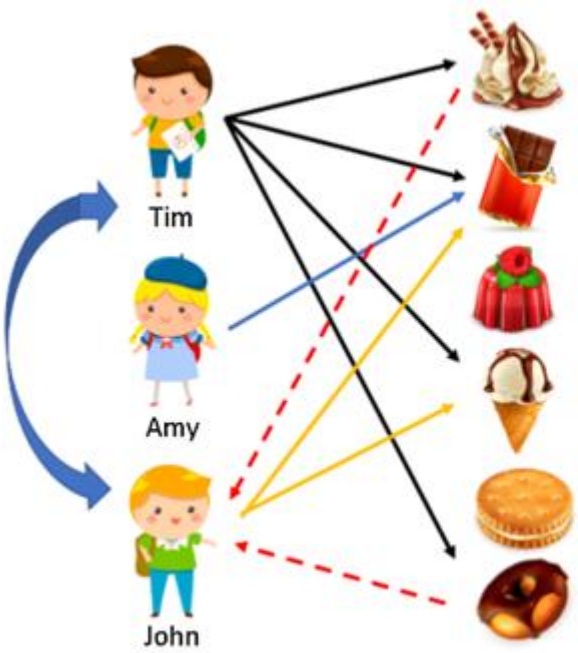


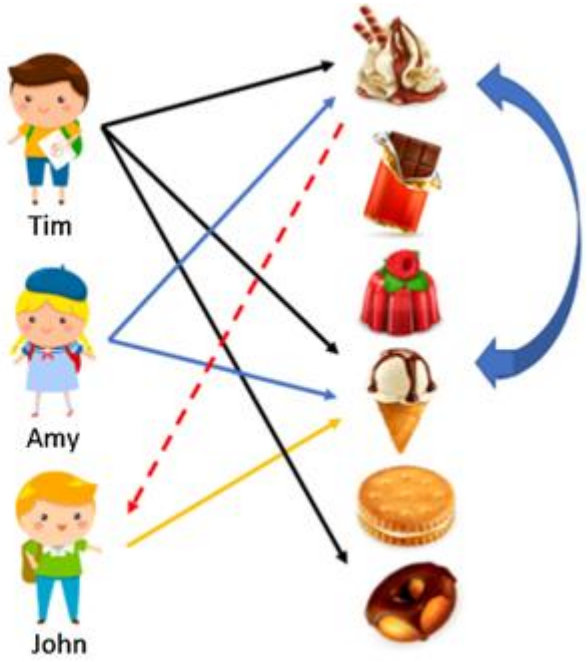
Rating matrix



Neighborhood based approach

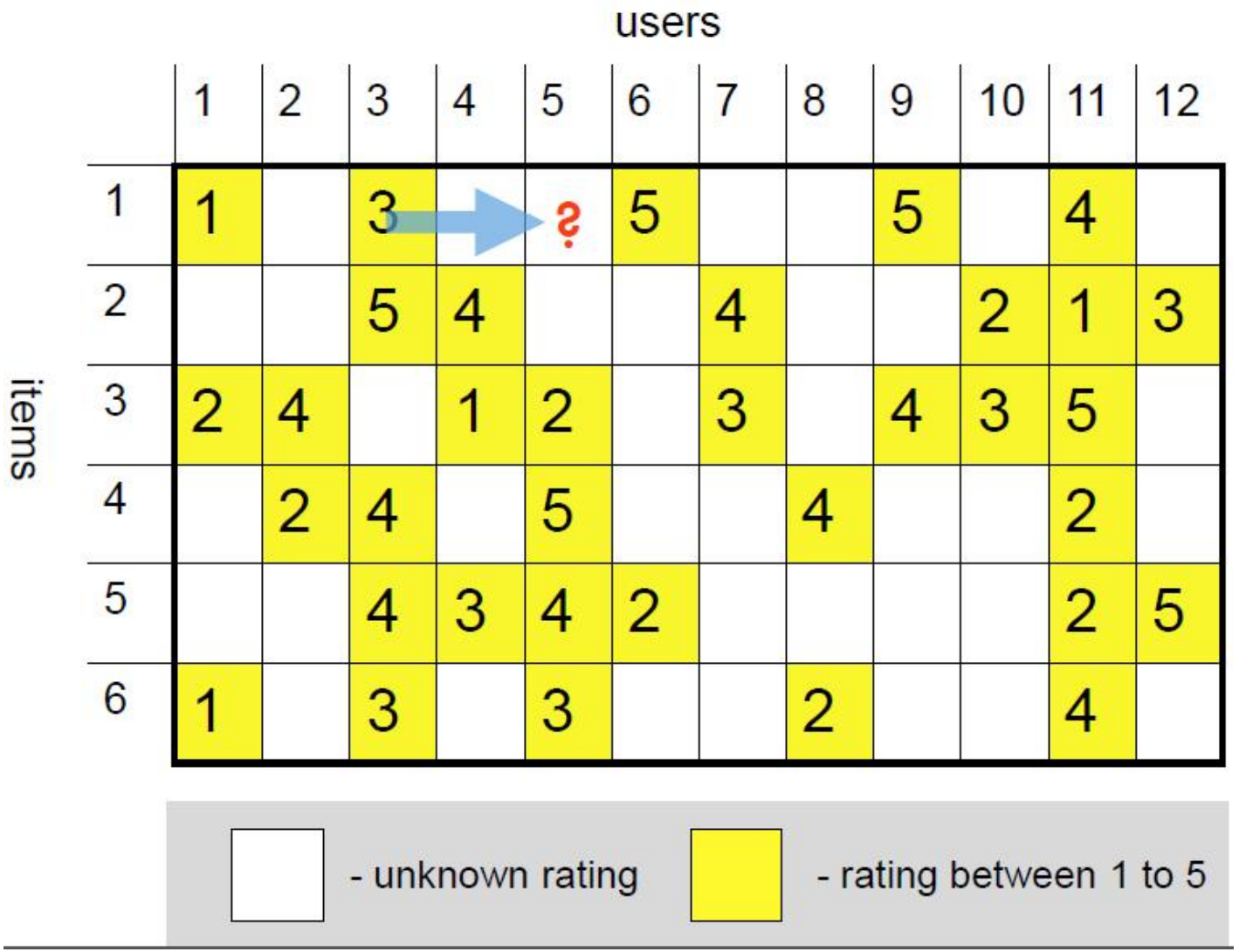
User based



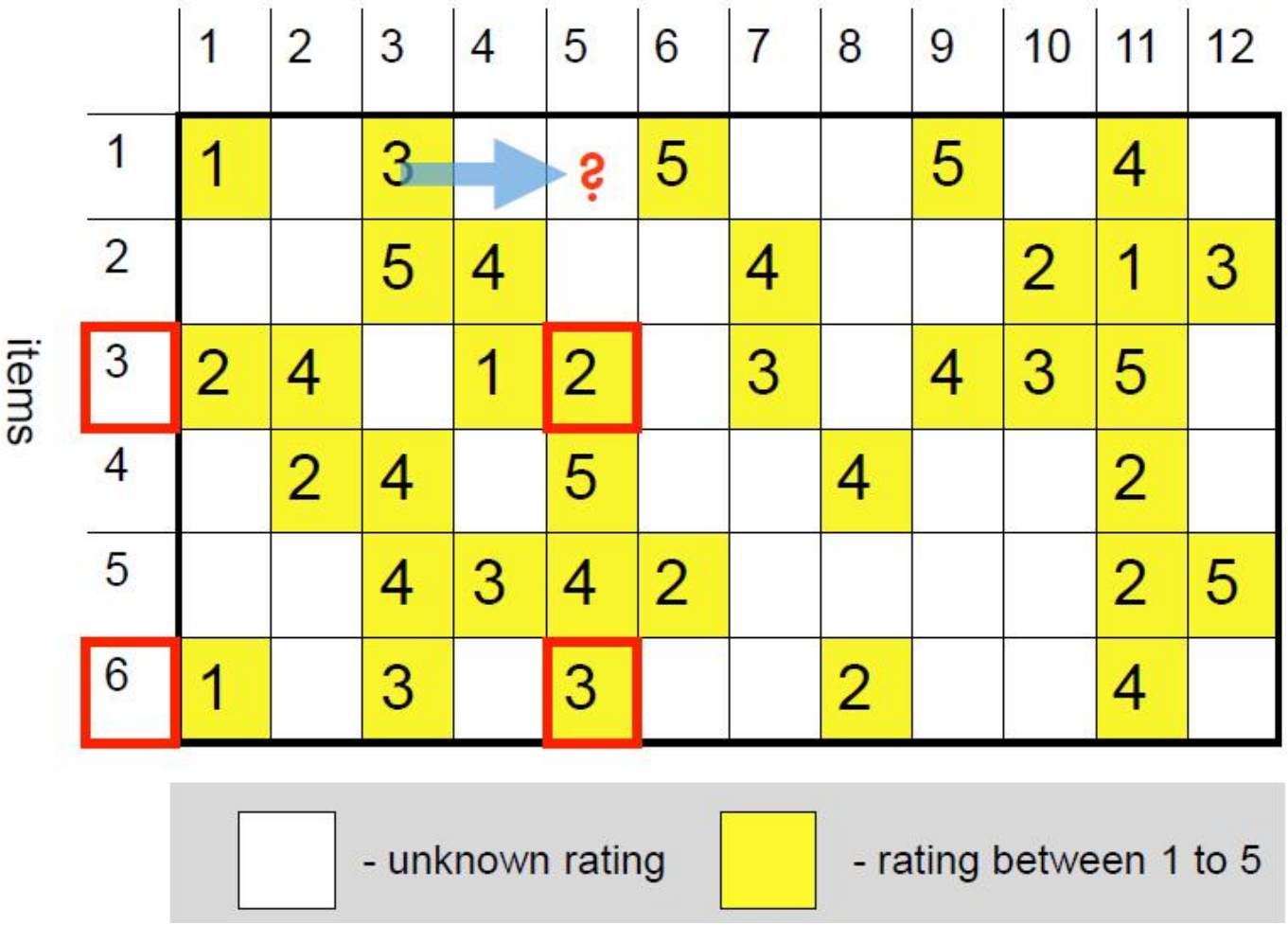


Item based

Neighborhood based approach



Neighborhood based approach



similarity

$s_{13} = 0.2$

$s_{16} = 0.3$

weighted average

$$\frac{0.2 \cdot 2 + 0.3 \cdot 3}{0.2 + 0.3} = 2.6$$

Similarity

- (user, user) similarity to recommend items
- (item, item) similarity to recommend new items that were also liked by the same users
- Reading: Amazon recommendation system at scale:

http://www.cs.umd.edu/~samir/498/Amazon-Recommendations.pdf

Measure Similarity

- How do we measure (user, user) similarity or (item, item) similarity?
  - Euclidean distance
  - Jaccard similarity
  - Cosine similarity
  - Pearson correlation

- User: indexed by u, item indexed by i
  - r_{ui}: rating(user, item)
  - U_i: set of users who purchased item i
  - I_u: set of items purchased by user u

- Focus on (item, item) similarity

Euclidean distance

- Distance between item i_1 and item i_2 is

$$\text{dist}^2(i_1, i_2) = \sum_u (r_{u,i_1} - r_{u,i_2})^2$$

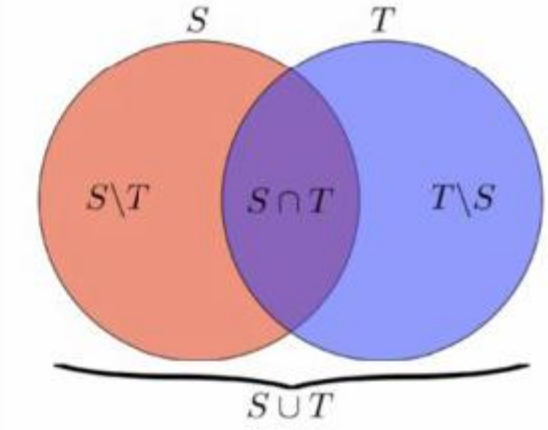
- If each rating is 0 or 1 (user bought item or did not), then the distance becomes

$$\begin{aligned}\text{dist}^2(i_1, i_2) =& \#\{\text{users that bought } i_1, \text{ but not } i_2\} \\ &+ \#\{\text{users that bought } i_2, \text{ but not } i_1\}\end{aligned}$$

Jaccard similarity

- Key idea: normalize by popularity

$$\text{Jaccard}(U_i, U_j) = \frac{|U_i \cap U_j|}{|U_i \cup U_j|} = \frac{\text{bought i and j}}{\text{bought i or j}}$$



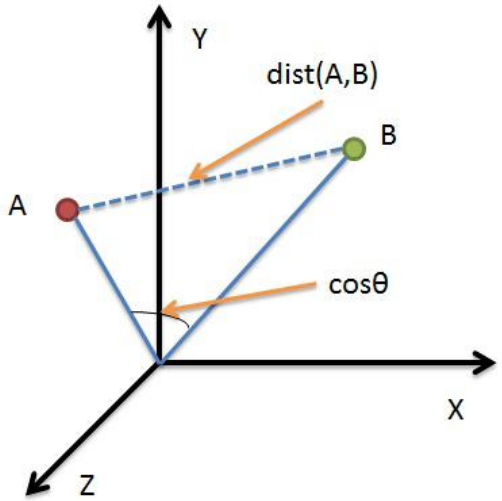
- Maximum is 1 if two items were purchased by the same set of users
- Minimum is 0 if the two items were purchased by completely disjoint sets of users

Example

- A user is looking at item i
- Goal: recommend him/her similar items:
- Let U_i is the set of users who viewed this product.
- Rank product j according to $\frac{|U_i \cap U_j|}{|U_i \cup U_j|}$

Cosine similarity

- Jaccard similarity works only on 0/1 data
- Cosine similarity works on arbitrary data.



$$\text{similarity}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

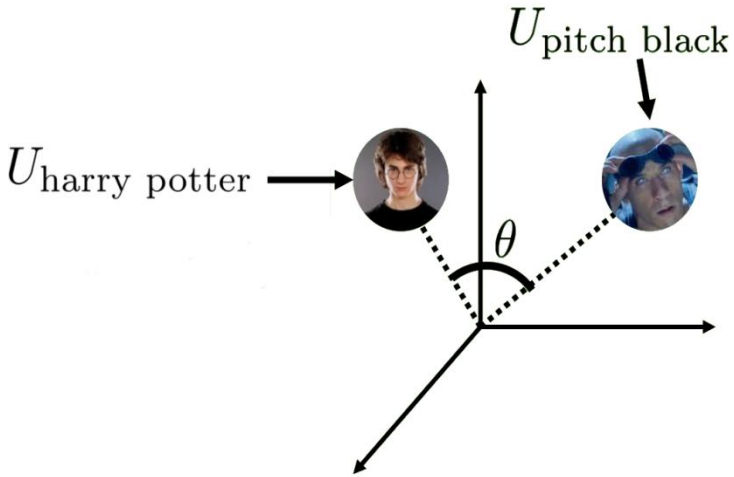
$\cos(\theta) = 1$ $(\theta = 0)$ A and B point in the same direction

$\cos(\theta) = -1$ $(\theta = 180)$ A and B point in the opposite direction

$\cos(\theta) = 0$ $(\theta = 90)$ A and B are orthogonal

Example

- Each item is represented by a vector of users' ratings



$$U_{\text{harry potter}} = (0, 1, 1) \qquad U_{\text{pitch black}} = (1, 1, 0)$$

$$\text{similarity} = \frac{(0, 1, 1) \cdot (1, 1, 0)}{\sqrt{2} \cdot \sqrt{2}} = \frac{1}{2}$$

- Jaccard: $\frac{|U_1 \cap U_2|}{|U_1 \cup U_2|} = \frac{1}{3}$

Pearson correlation

- Pearson correlation: removing average from the rating vector

User ratings for item i:

1	?	?	5	5	3	?	?	?	4	2	?	?	?	?	4	?	5	4	1	?

User ratings for item j:

?	?	4	2	5	?	?	1	2	5	?	?	2	?	?	3	?	?	?	5	4

- Pearson correlation computed over shared support

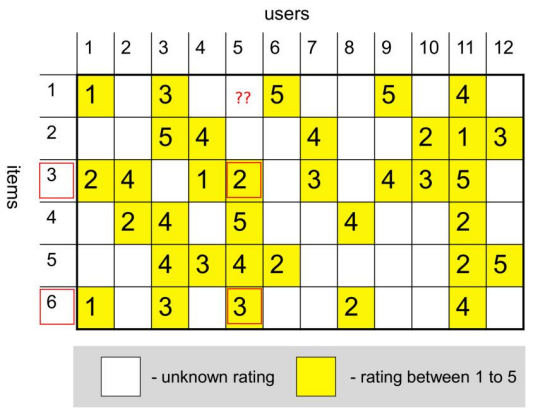
$$s_{ij} = \frac{\sum_{u \in U_i \cap U_j} (r_{ui} - \bar{r}_i)(r_{uj} - \bar{r}_j)}{\sqrt{\sum_{u \in U_i \cap U_j} (r_{ui} - \bar{r}_i)^2 \cdot \sum_{u \in U_i \cap U_j} (r_{uj} - \bar{r}_j)^2}}$$

- $\bar{r}_i = \frac{\sum_{u \in U_i \cap U_j} r_{ui}}{|U_i \cap U_j|}$
- In this example, $\bar{r}_i = 3.8$, $\bar{r}_j = 4$, $s_{ij} = -0.43$

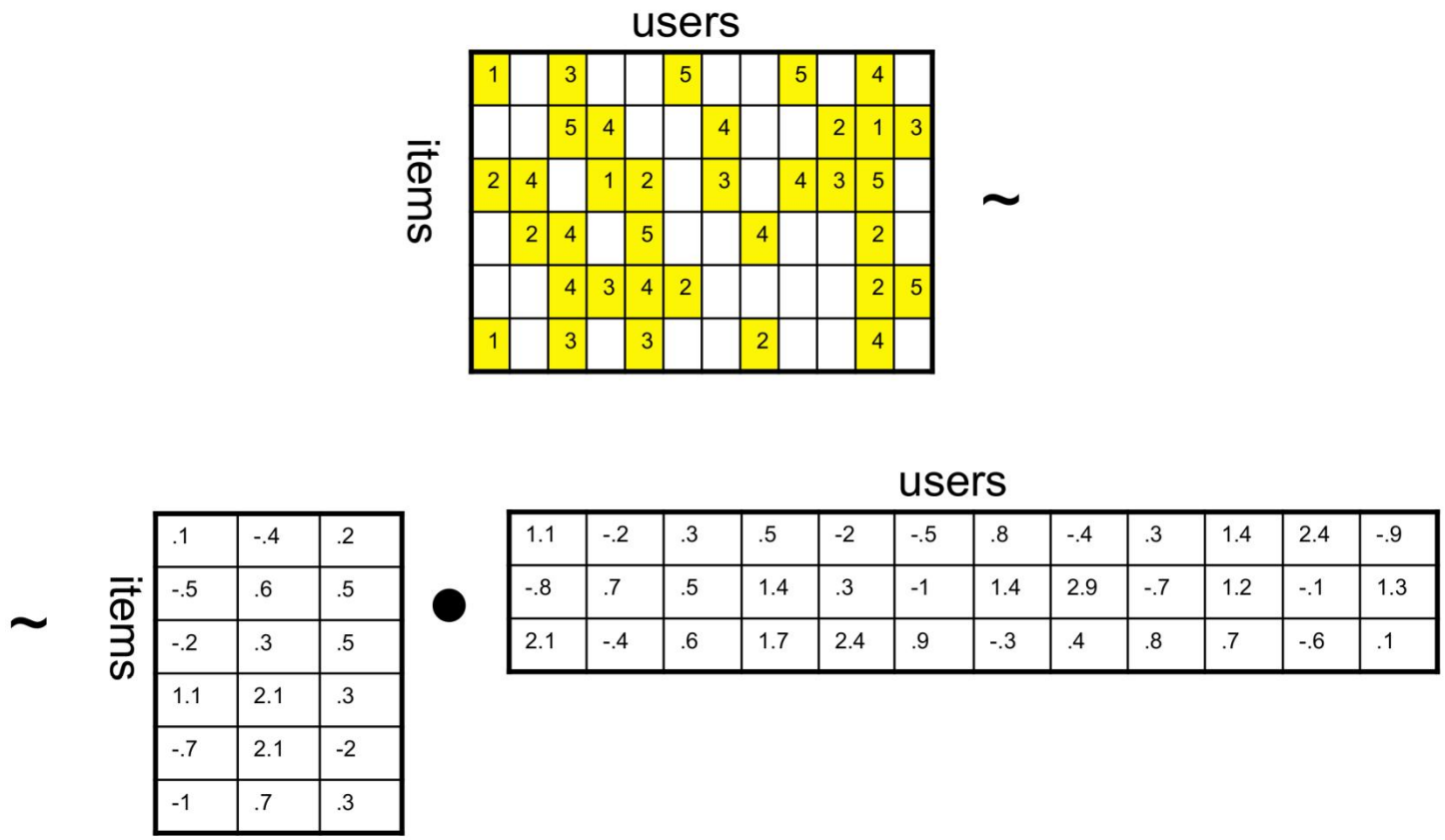
Summarize

- Given similarity measure between items s_{ij}
- Find set $s_k(i, u)$ of k-nearest neighbors to item i that were rated by user u
- Estimate rating using weighted average over the set of neighbors

$$\hat{r}_{ui} = \frac{\sum_{j \in s_k(i,u)} s_{ij} r_{uj}}{\sum_{j \in s_k(i,u)} s_{ij}}$$



Latent factor model



Latent factor model

- Each movie and user can be characterized by k-dimensional vector
- De How much is it action, romance, drama, . . .

$$q_i = (0.9, 0.2, 0.5, \ldots)$$

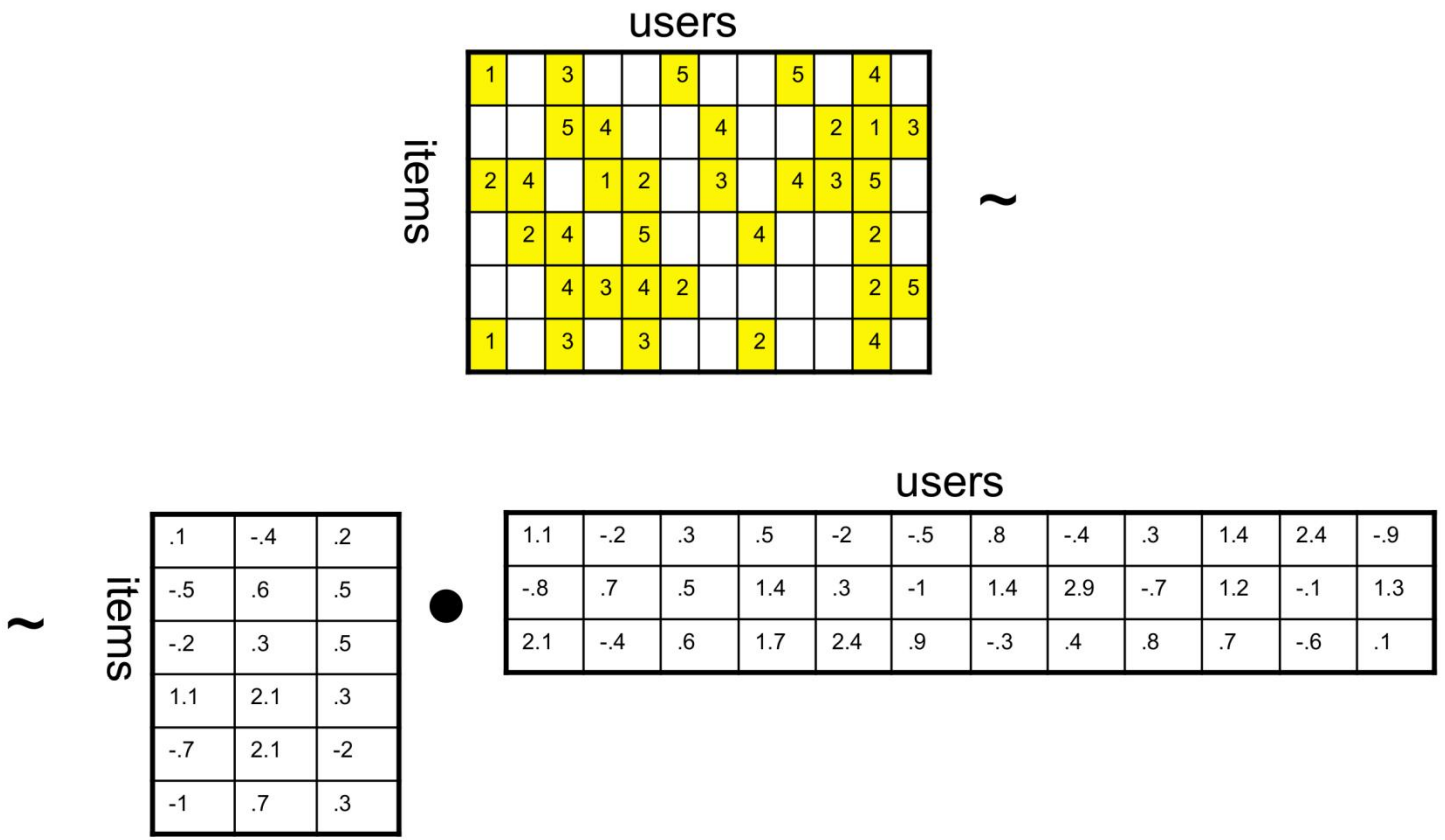
- Describ How much she likes action, romance, drama, . . .

$$p_u = (0.01, 0, 0.9, \ldots)$$

- $f(u, i) = p_u \cdot q_i$ is the product of the two vectors
  - $f(u, i) = 0.9 \cdot 0.01 + 0.2 \cdot 0 + 0.5 \cdot 0.9 + \ldots$

How to discover the features

- Minimize the distance to the observed ratings



1		3			5			5		4	
		5	4			4			2	1	3
2	4		1	2		3		4	3	5	
	2	4		5			4			2	
		4	3	4	2					2	5
1		3		3			2			4	

.1	-.4	.2
-.5	.6	.5
-.2	.3	.5
1.1	2.1	.3
-.7	2.1	-2
-1	.7	.3

1.1	-.2	.3	.5	-2	-.5	.8	-.4	.3	1.4	2.4	-.9
-.8	.7	.5	1.4	.3	-1	1.4	2.9	-.7	1.2	-.1	1.3
2.1	-.4	.6	1.7	2.4	.9	-.3	.4	.8	.7	-.6	.1

An optimization problem

- Model $\hat{r}_{ui} = \boldsymbol{p}_u^T \cdot \boldsymbol{q}_i$
- Minimizing the following objective function

$$\min_{\boldsymbol{p},\boldsymbol{q}} \sum_{(u,i):r_{ui}\neq ?} (r_{ui} - \hat{r}_{ui})^2 + \lambda\left(\sum_u \|\boldsymbol{p}_u\|^2 + \sum_i \|\boldsymbol{q}_i\|^2\right)$$

L2 **regularization**

- Using gradient descent algorithm

An optimization problem

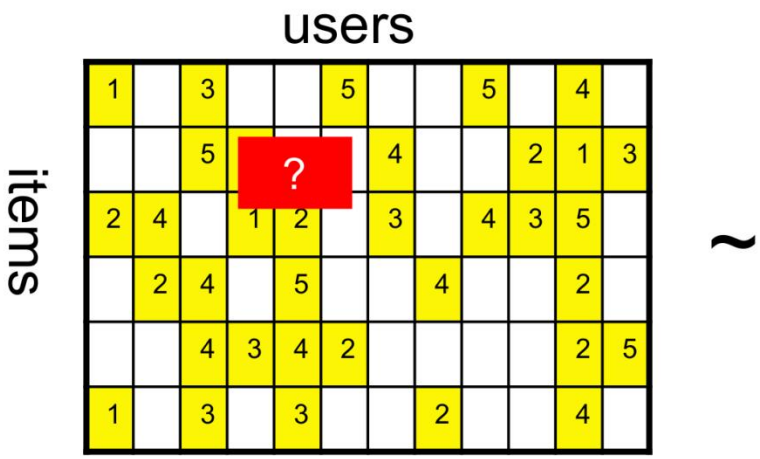
- Basic-SVD Model $\hat{r}_{ui} = \boldsymbol{p}_u^T \cdot \boldsymbol{q}_i + b_u + b_i + b$
- Minimizing the following objective function

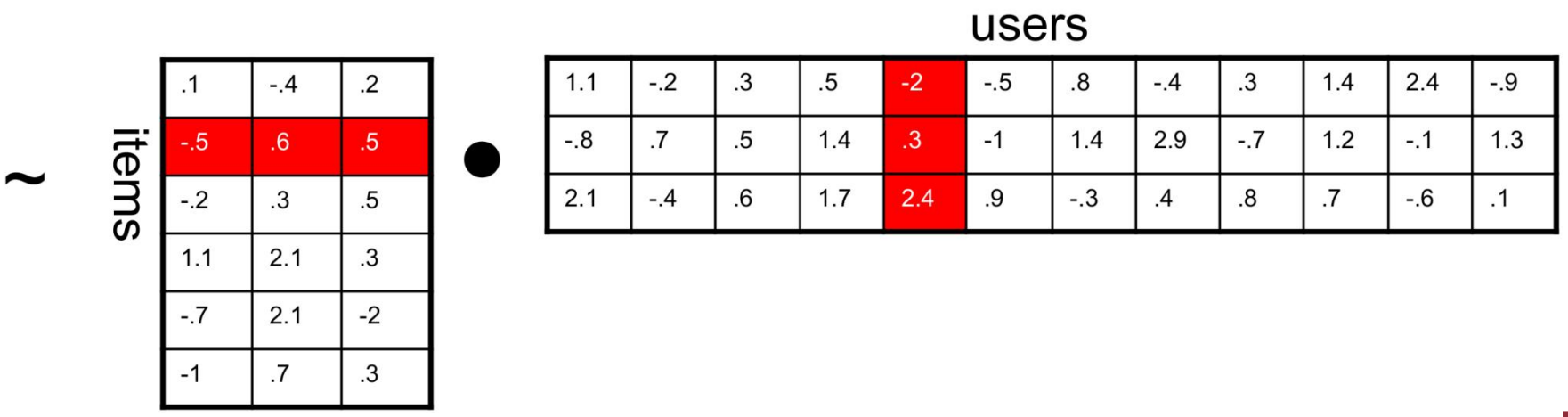
$$\min_{\boldsymbol{p},\boldsymbol{q}} \sum_{(u,i):r_{ui}\neq ?} (r_{ui} - \hat{r}_{ui})^2 + \lambda\left(\sum_u \|\boldsymbol{p}_u\|^2 + \sum_i \|\boldsymbol{q}_i\|^2 + b_u^2 + b_i^2\right)$$

L2 **regularization**

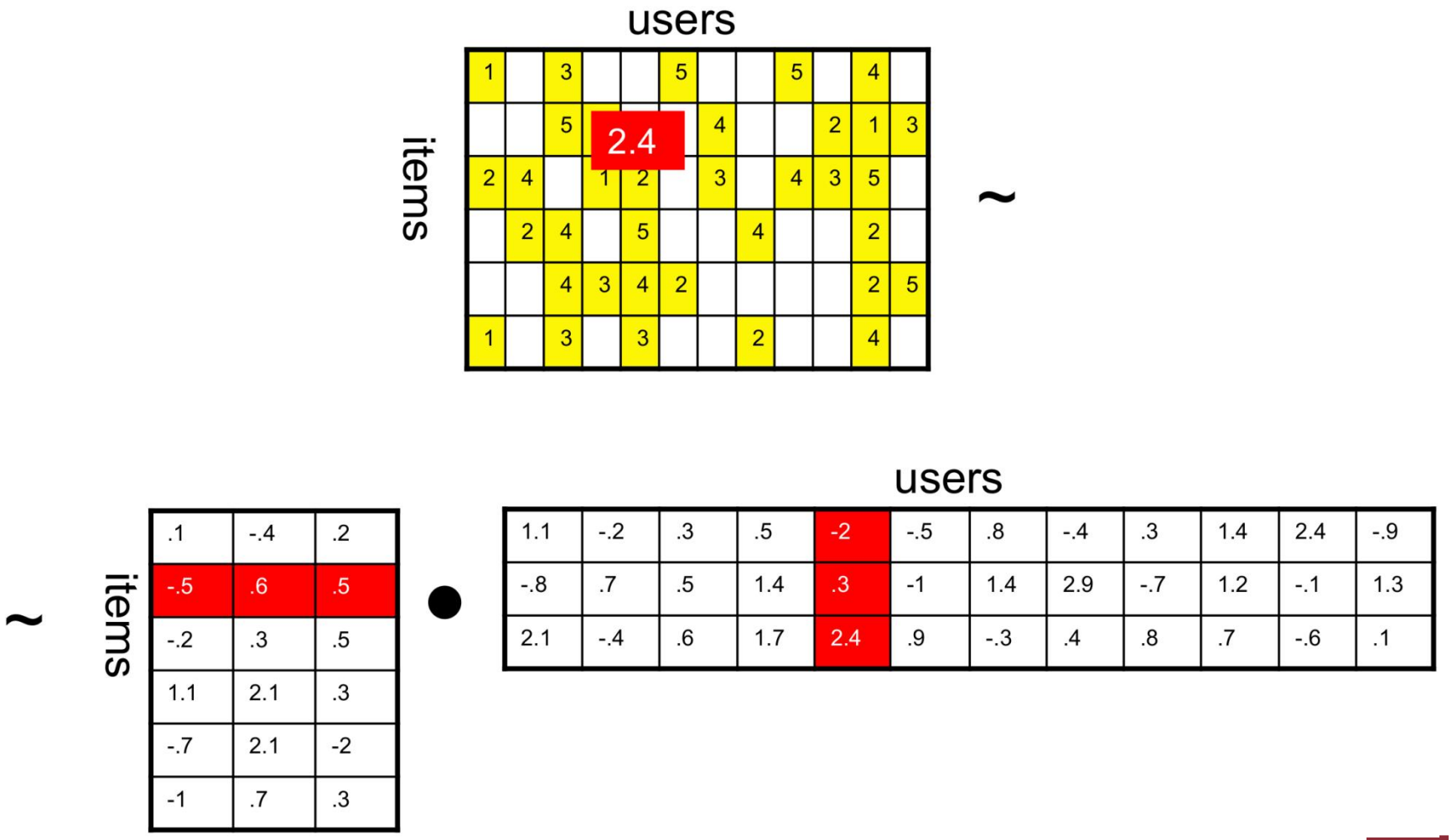
- Using gradient descent algorithm

Prediction





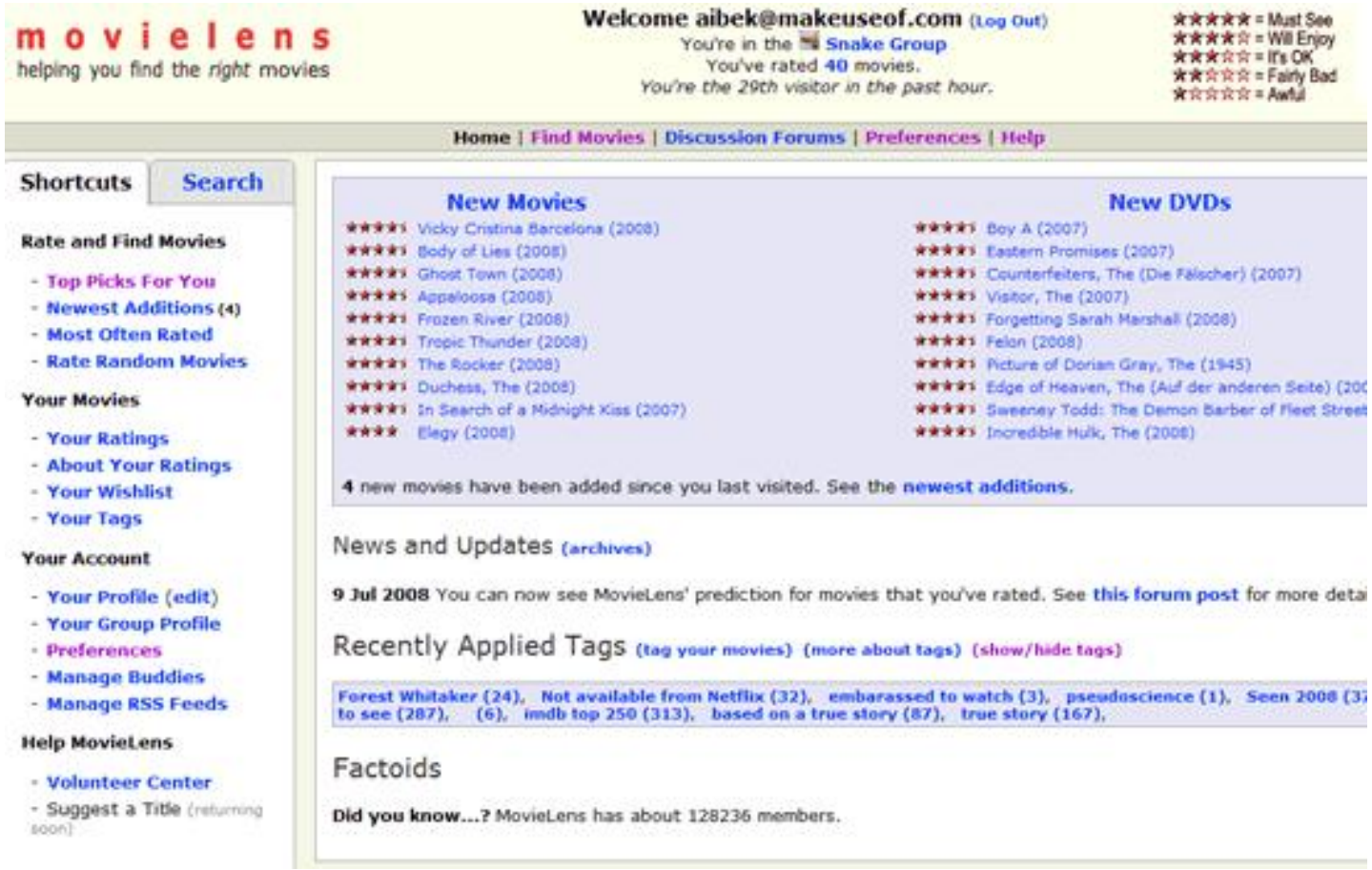
Prediction


users
items
2.4
~
~
items
users
.1 -.4 .2
-.5 .6 .5
-.2 .3 .5
1.1 2.1 .3
-.7 2.1 -2
-1 .7 .3
1.1 -.2 .3 .5 -2 -.5 .8 -.4 .3 1.4 2.4 -.9
-.8 .7 .5 1.4 .3 -1 1.4 2.9 -.7 1.2 -.1 1.3
2.1 -.4 .6 1.7 2.4 .9 -.3 .4 .8 .7 -.6 .1

Outline

- What is recommender system
- Content-based method
- Collaborative filtering method
  - User based nearest neighbor
  - Item based nearest neighbor
  - Latent factor model
- Case study

Case study: MovieLens data



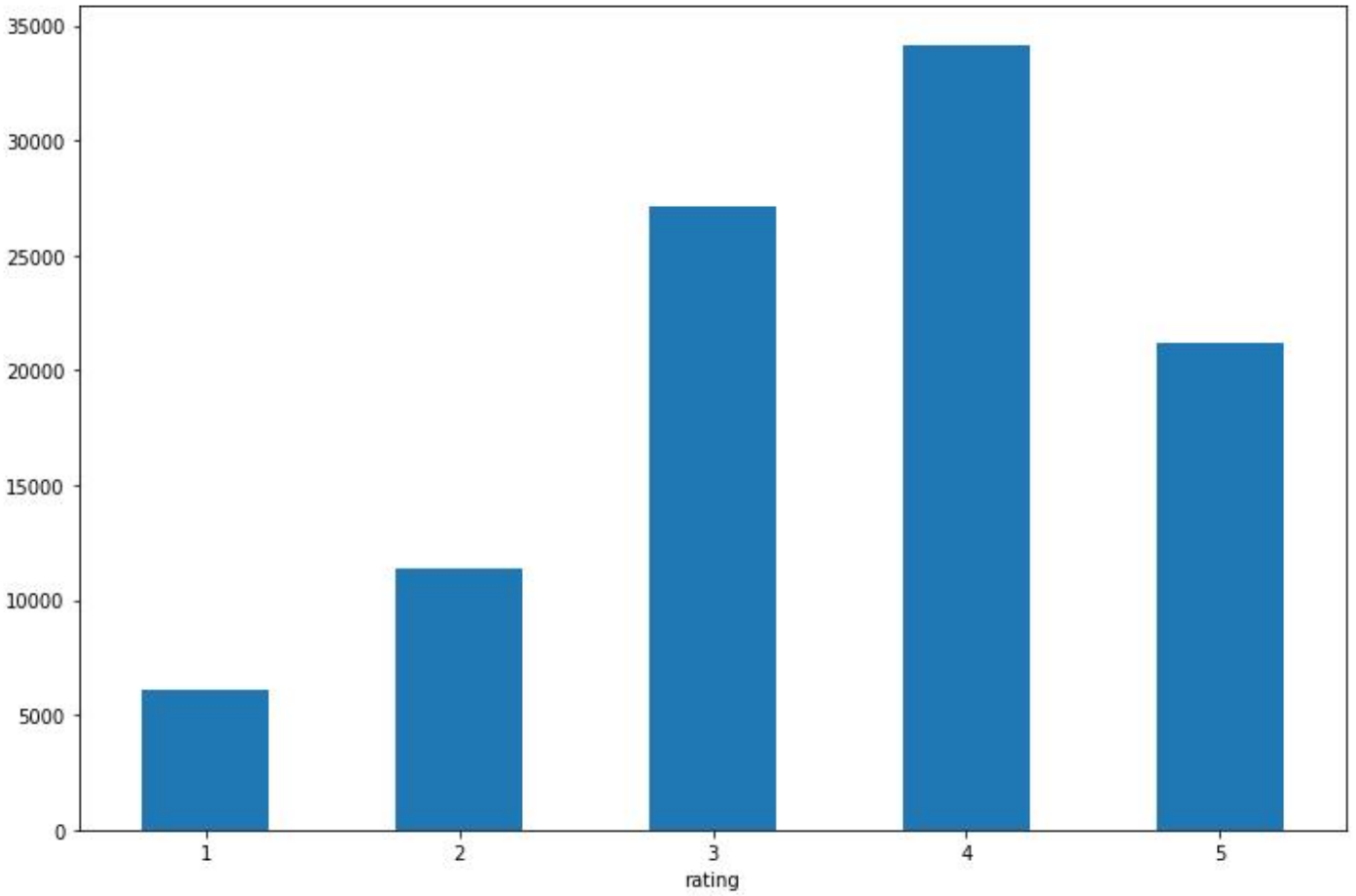
Data Information

- **MovieLens 100k**
  - 100,000 ratings
  - 943 users
  - 1683 films

- **Data Source:**
  - http://grouplens.org/datasets/movielens/

- **# Data structure:**
  - user id,
  - item id,
  - rating,
  - rating time

Ratings



```
# 5.2.1.SVD: (biased=True)
algo = SVD(random_state=1)
algo.fit(trainset)
predictions = algo.test(testset)

rmse['SVD'] = accuracy.rmse(predictions)
mae['SVD'] = accuracy.mae(predictions)
```

```
RMSE: 0.9454
MAE:  0.7463
```

```
# 5.2.2.MF: (biased=False)
algo = SVD(biased=False, random_state=1)
algo.fit(trainset)
predictions = algo.test(testset)

rmse['MF'] = accuracy.rmse(predictions)
mae['MF'] = accuracy.mae(predictions)
```

```
RMSE: 0.9617
MAE:  0.7595
```

Some Takeaways

- Content based recommendation
- CF method: NN
- CF method: latent factor model

Acknowledgement

- NYU: machine learning course (by Xi Chen)
- UW: Introduction to machine learning (by Jeff Howbert)
- Chicago Booth Course: Machine learning (by Carlos Carvalho, Mladen Kolar and Robert McCulloch) [MovieLens Data Example]
- Dietmar Jannach et al., Recommender Systems – An Introduction

The End

THANK YOU

Any questions?